COS214 - PROJECT
GIT, PUSH AND PRAY

GitHub Repository Link: https://github.com/v-rex401/TIVIDY_Project



Team Details:

- Vashti Pillay u25041887
- Anke de Frey u24611400
- Kayla Falconer u25006747
- Jean-Pierre Joubert u25056809
- Jezelle Govender u25176383
- Azolile Mbanga u24820360
- Ronin Sydney u25056507

Task 1: Research Workflow Management Systems

A workflow is a structured sequence of tasks that are carried out in a particular order to achieve an outcome. A Workflow Management System manages this process by coordinating tasks, considering their dependencies and directing work to the appropriate people or system [8]. From looking at different workflow systems, it became clear that a workflow management system is more than just a list of tasks. It needs to manage how work is created, assigned, routed, completed and monitored throughout the process.

One of the most important concepts identified during the research was the difference between a workflow definition and a workflow instance. A workflow definition is the reusable blueprint of a process. It describes the tasks, actions, transitions and overall structure of the workflow. A workflow instance is one actual execution of that definition and contains its own runtime information, such as its current state and progress [7]. This distinction is important for TIVIDY because the same workflow definition should be able to create multiple independent instances at the same time.

The research also showed that certain responsibilities appear repeatedly across different workflow systems. These include managing workflow, definitions and instances, creating and assigning work items, routing work between activities, responding to events and dependencies, recording workflow history, monitoring execution and communicating with external systems. Workflow systems can also contain nested subprocesses, where one activity contains several smaller activities, gateways and events [4].

Another important finding was that some parts of the system are expected to change independently. For example, the way work is assigned may depend on roles, workload or other rules. Approval and escalation processes may also differ between organisations. Decisions such as routing, assignment and approval can be kept as configurable rules instead of being hard coded into the workflow engine [1]. This suggests that our system should keep these changing behaviours separate from its core workflow execution.

External systems are another area that needs to remain flexible. Workflow systems can communicate with other systems using connectors, API's and messaging [1]. For TIVIDY, this means that the workflow manager should coordinate with external systems without becoming responsible for the functionality of those systems. The research also showed that workflow definitions may change while existing workflows are still running. AWS Simple Workflow Service handles this using versioning, allowing existing instances to continue with their original definition while new instances use the newer version [5].

Overall, the research helped identify what TIVIDY actually needs to be designed around. TIVIDY should be a generic and reusable workflow manager, rather than a system containing logic for one specific process. It needs to manage workflow execution, work items, routing, participants, state, events, history and external integrations. At the same time, behaviours such as routing strategies, approval rules, notifications and external system interfaces should be flexible enough to change without changing the core workflow engine.

References

[1] Connector types. (n.d.). Camunda 8 Docs. Retrieved October 2, 2026, from https://docs.camunda.io/docs/components/connectors/connector-types/

[2] Defining a workflow action. (n.d.). IBM. Retrieved October 2, 2026, from https://www.ibm.com/docs/en/openpages/9.2.x?topic=actions-defining-workflow-action

[3] Designing a workflow. (n.d.). IBM. Retrieved October 2, 2026, from https://www.ibm.com/docs/en/openpages/9.2.x?topic=workflows-designing-workflow

[4] Subprocess. (n.d.). Flowable Enterprise Documentation. Retrieved October 2, 2026, from https://documentation.flowable.com/latest/reactmodel/bpmn/reference/subprocess

[5] Versioning - Amazon Simple Workflow Service. (n.d.). AWS Documentation. Retrieved October 2, 2026, from https://docs.aws.amazon.com/amazonswf/latest/developerguide/swf-dev-adv-versioning.html

[6] What Is a Workflow? (n.d.). IBM. Retrieved October 2, 2026, from https://www.ibm.com/think/topics/workflow

[7] Workflow definitions and workflow instances. (n.d.). IBM. Retrieved October 1, 2026, from https://www.ibm.com/docs/en/openpages/9.2.x?topic=fundamentals-workflow-definitions-workflow-instances

[8] Workflow Management System - Glossary | CSRC. (n.d.). NIST Computer Security Resource Center. Retrieved October 2, 2026, from https://csrc.nist.gov/glossary/term/workflow_management_system

Task 2: Define the Scenario and System

1. Initial Scope of TIVIDY
   TIVIDY is a generic workflow management system that coordinates workflows
   Its main responsibilities are to:
   - Store and manage workflow definitions and versions
   - Create and manage workflow instances
   - Create, assign and track work items
   - Route work using configurable rules
   - Manage participants, approvals, reactions and escalation
   - Handle states, events and dependencies
   - Communicate with external systems
   - Record history and audit information
   - Monitor running workflows

2. System Boundaries
   TIVIDY is responsible for coordinating and tracking work, but not for performing the specialised business work itself.
   For example: TIVIDY can assign a doctor an assessment task and track its completion. It also does not replace the hospital's external systems.

3. Workflow definition vs workflow instance
   Workflow definition: reusable blue print of a process. It contains activities, transitions, routing rules, dependencies, approval and escalation rules, events and its version.
   A workflow instance: represents one actual execution of a definition. It contains runtime information such as its current activity and state, assigned participants, completed or pending work, events, errors and history.
   This means that multiple independent instances can run from the same definition.

4. Parts expected to vary
   The following parts should be able to change independently:
   - Assignment and routing: eg. role based, workload based or rule based.
   - Approval and escalation rules: different processes may require different approval levels.
   - Work item behaviour: validation, priority, notifications and security.
   - External integrations: different systems may require different interfaces.
   - Reporting: organisations may require different workflow statistics and reports.

5. Workflow scenario #1: Hospital Patient Admissions and Treatment

Organisation/Environment

The scenario represents a hospital environment where different departments and roles must coordinate a patient's admission, assessment, treatment and discharge.

Process

The Hospital workflow uses a Sequential builder to construct a definition containing stages representing a patient's journey containing 5 stages: Admission, Assessment, Emergency Care, Treatment, and Discharge.

Participants/Roles

- Admission staff: register the patient and capture information
- Nurse: perform the initial assessment and record observations
- Doctor: assess the patient and determine treatment
- Senior Doctor/Specialist: handles escalated cases(Emergency care) and higher risk approvals

Our configured staff members use a rank in the staff escalation chain.

Major Work

1. Admission: Register Patient; Capture patient details
2. Assessment: Nurse assessment; Doctor assessment; request tests
3. Emergency Care: Stabilise patient
4. Treatment: Determine treatment; obtain approval; perform treatment
5. Discharge: Monitor patient; discharge patient

Decisions and Routing

- Approval-chain routing: The approval limit is set at 6. The standard destination is the Treatment stage, while cases requiring escalation are directed to Emergency Care.
- Event-based routing: The initial input data contains a triage score of 3. Routing thresholds are configured so that values of 8 or higher are directed to Emergency care, values of 4 or higher to Treatment, and values below 4 to Assessment.

These strategies allow work to be directed to different stages depending on the selected routing strategy and the relevant approval or event value.

Approvals and Dependencies

The staff members are linked by a chain of escalation, starting at Admission Clerk, through Nurse and Doctor to Senior Doctor. When a staff member cannot handle an approval request because their rank is insufficient, the request can be passed to the next staff member in the chain. The workflow definition also groups the work items into 5 stages, with the Sequential builder linking the stages in their defined order.

Failures/Exceptions

The approval escalation is the main exception path demonstrated by this scenario. If a staff member does not have the authority to handle an approval request, the request is forwarded to a more senior staff member. If the request reaches a staff member with sufficient authority, that staff member can handle it. The scenario also supports alternative routing through the configured event thresholds, allowing work to be directed to Emergency care or another stage depending on the event value.

External System

- Notification system: TIVIDY can communicate with the notification system through the NotificationAdapter. The WorkflowManager uses the external system interface when work is assigned or routed.

TIVIDY coordinates these interactions through its external system interface.

Expected Outcome

The Hospital scenario displays how TIVIDY can construct a hospital workflow definition, organise work into stage, assign work to staff members and route work using configurable strategies. It also demonstrates how approval requests can be escalated through a staff hierarchy and how workflow instances can have their state changes monitored.

6. Workflow scenario #2: Insurance Claim

Organisation/Environment

This scenario represents an insurance company that manages claims submitted by clients. Different staff members are responsible for capturing claims, assessing them, reviewing larger claims and processing settlements.

Process

The Insurance Claim workflow uses an Escalating builder to construct a definition containing 4 stages: Lodgement -> Assessment -> Senior Review -> Settlement
Each Stage contains work items representing the activities assigned to the relevant staff roles.

Participants/Roles

- Claims clerk: Responsible for capturing the claim and checking policy cover
- Assessor: Responsible for inspecting the damage and estimating the repair cost
- Senior assessor: Responsible for reviewing larger claims that require escalation
- Claims manager: Responsible for approving payouts and paying the claimant

Major Work

1. Lodgement: Capture claim; check policy cover
2. Assessment: Inspect damage; estimate repair cost
3. Senior review: Review large claim

4. Settlement: Approve payout; pay claimant

Decisions and Routing

- Approval-chain routing: The approval limit is set as 50,000. The standard destination is Assessment, while the requests that require escalation are directed to Senior review.
- Event-based routing: The initial input data contains a claim amount of 12,000. Routing thresholds are configured so that values of 100,000 or higher are directed to Senior review, of 20,000 or higher to Assessment, and values below 20,000 to Lodgement.

These strategies allow work to be directed to different stages depending on the selected routing strategy and the relevant approval or claim amount.

Approvals and Dependencies

The staff members are arranged in an escalation chain, from Claims clerk through Assessor and Senior assessor to Claims manager. When a staff member cannot handle an approval request because their rank is insufficient, the request can be passed to the next staff member in the chain.
The workflow definition also organises the claim activities into the 4 stages, with the Escalating builder constructing the workflow using its escalating arrangement.

Failures/Exceptions

Approval escalation is the main exception path demonstrated by this scenario. If a staff member does not have the authority to handle an approval request, the request can be forwarded to a more senior staff member. If the request reaches a staff member with sufficient authority, that staff member can handle it.
The scenario also supports alternative routing through the configured event thresholds. A claim amount reaching 100,000, for example, meets the threshold for Senior review.

External Systems

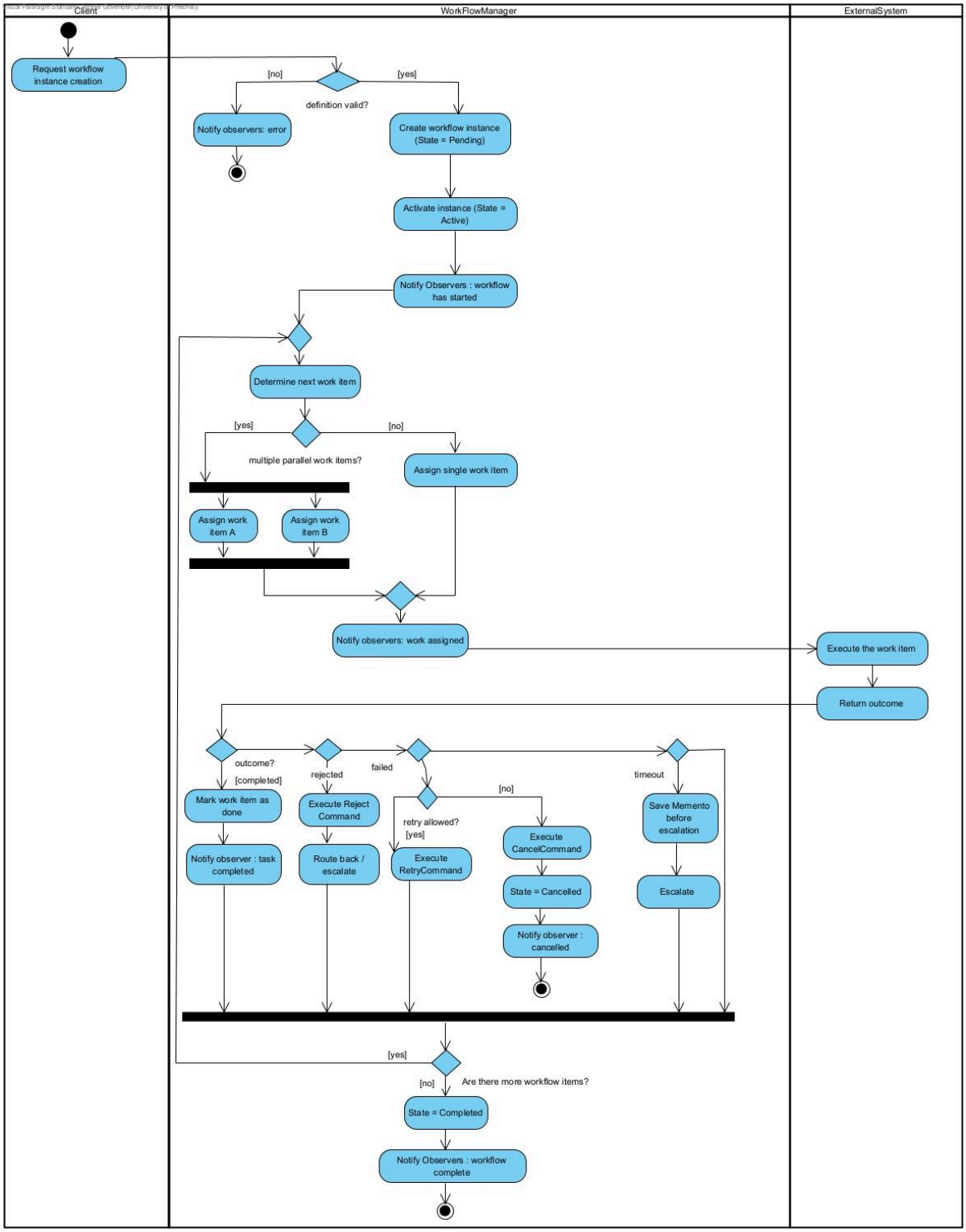
- Notification system: TIVIDY can communicate with the notification system through the NotificationAdapter. The WorkflowManager uses the external system interface when work is assigned or routed.

TIVIDY coordinates these interactions through its external system interface.

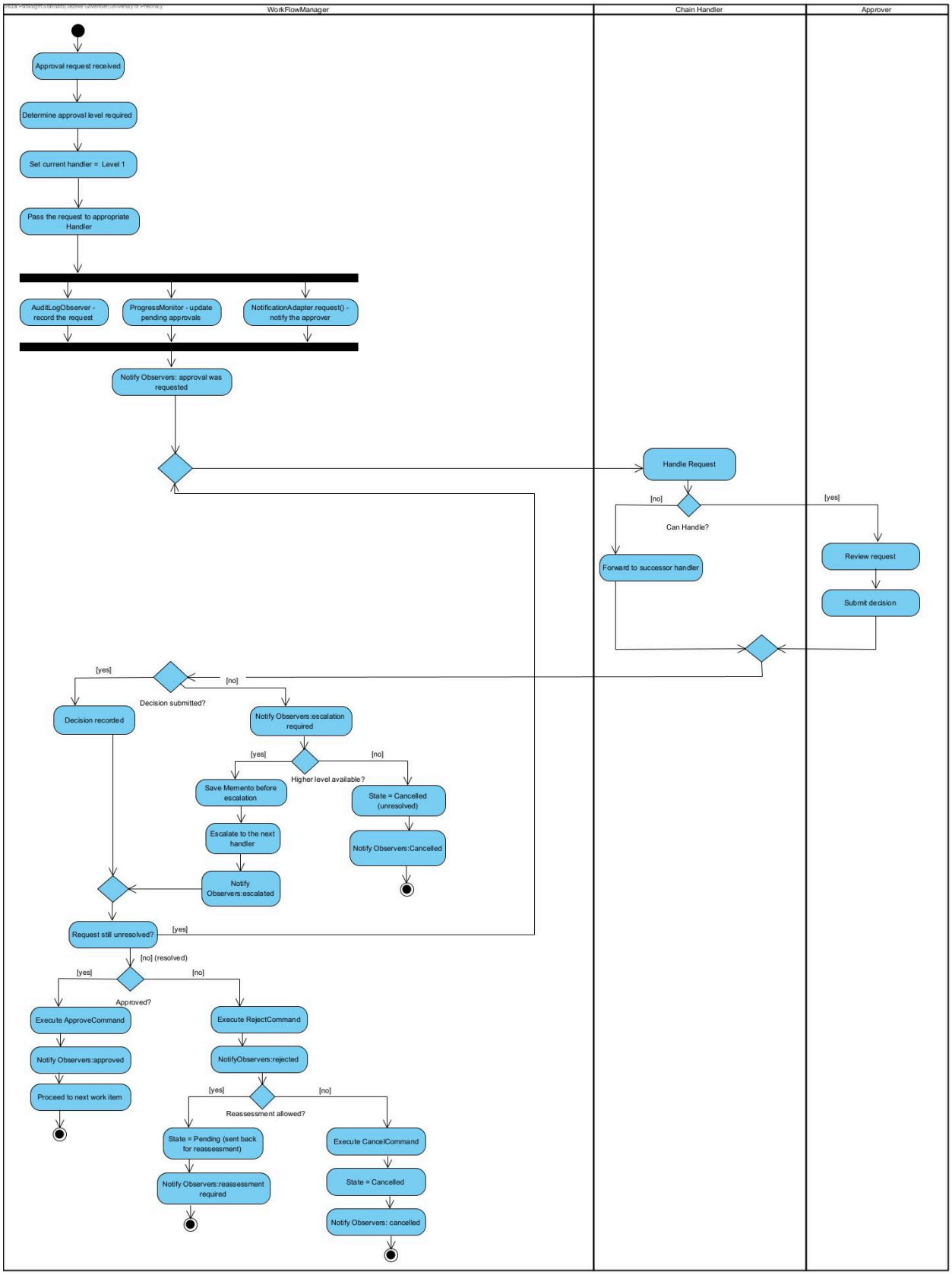
Expected Outcome

The Insurance Claim scenario demonstrates how TIVIDY can construct an insurance workflow definition, organise claim activities into stages, assign work to staff members and route work using configurable strategies. It also demonstrates how approval requests can be escalated through a staff hierarchy and how claim amounts can be used as event data to determine routing destinations. Workflow instances can have their state changes monitored.

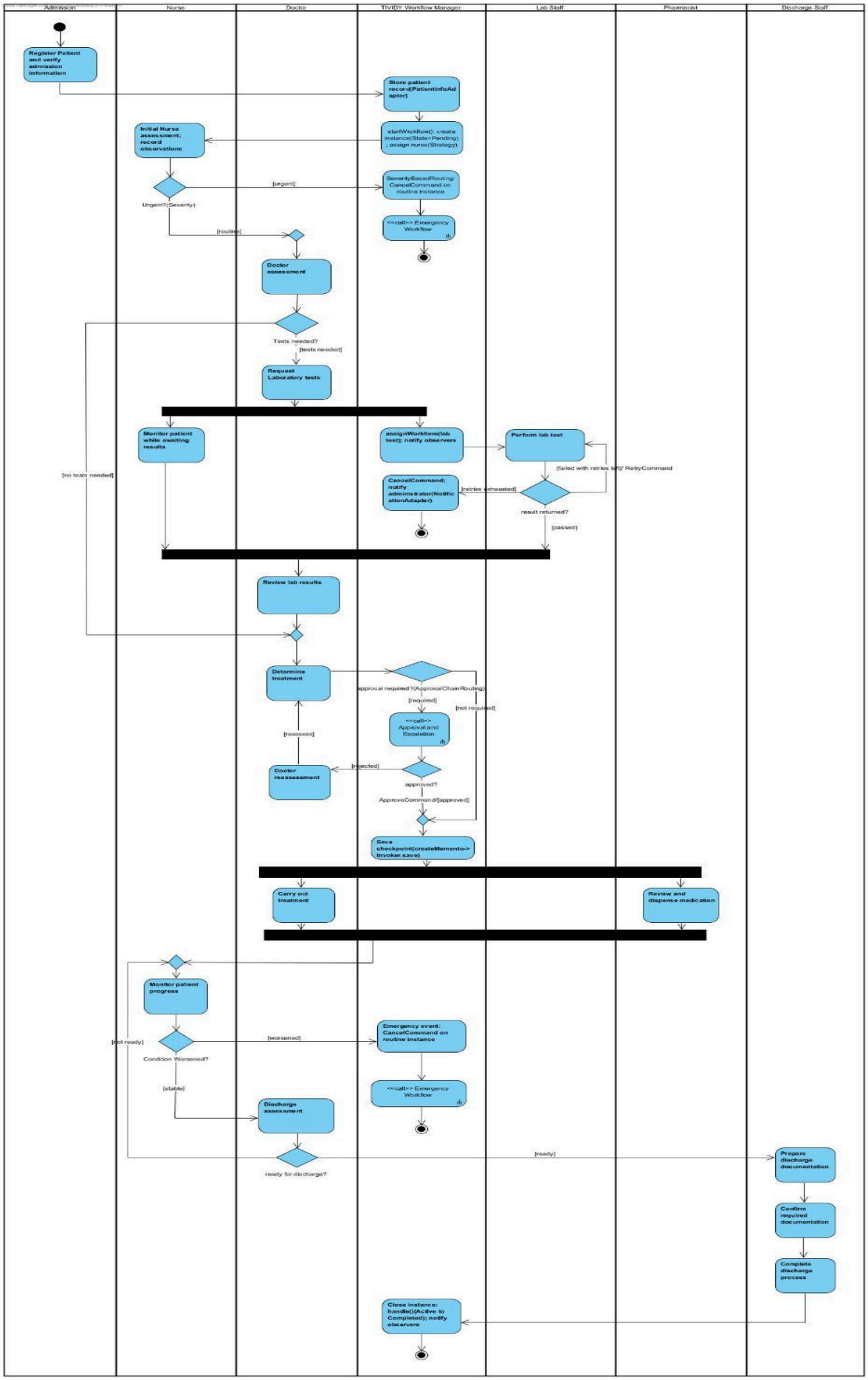
Task 3: UML Activity Diagrams



Activity Diagram 1- Execution from Creation to Completion



Activity Diagram 2- approval and escalation


Admission
Nurse
Doctor
TIVIDY Workflow Manager
Lab Staff
Pharmacist
Discharge Staff
Register Patient and verify admission information
Store patient record(PatientInfoAdapter)
startWorkflow(); create instance(State=Pending); assign nurse(Strategy)
Initial Nurse assessment; record observations
[urgent]
SeverityBasedRouting: CancelCommand on routine instance
Urgent?(Severity)
<<call>> Emergency Workflow
[routine]
Doctor assessment
Tests needed?
[tests needed]
Request Laboratory tests
Monitor patient while awaiting results
assignWorkItem(lab test); notify observers
Perform lab test
[failed with retries left]/ RetryCommand
CancelCommand; notify administrator(NotificationAdapter)
[retries exhausted]
result returned?
[passed]
[no tests needed]
Review lab results
Determine treatment
approval required?(ApprovalChainRouting)
[required]
[not required]
<<call>> Approval and Escalation
[reassess]
Doctor reassessment
[rejected]
approved?
ApproveCommand/[approved]
Save checkpoint(createMemento->Invoker.save)
Carry out treatment
Review and dispense medication
Monitor patient progress
[not ready]
Condition Worsened?
[worsened]
Emergency event: CancelCommand on routine instance
<<call>> Emergency Workflow
[stable]
Discharge assessment
ready for discharge?
[ready]
Prepare discharge documentation
Confirm required documentation
Complete discharge process
Close instance: handle()(Active to Completed); notify observers

Activity Diagram 3- Hospital Scenario(Patient Admission, Treatment Approval, Monitoring and Discharge)

Task 4: Design Patterns

(1) Adapter - TIVIDY needs integration with external systems and this design pattern converts the interface of an external class into another interface that the client expects, allowing the incompatible interfaces to work together without modifying their source code. *ExternalSystem(Target)* defines the interface the client expects. *NotificationSystem(Adaptee)* represents the notification service. *NotificationAdapter(Adapter)* translates the expected *request()* call into *sendNotification()* on that service. *WorkflowManager(Client)* communicates with the external system through the abstract interface. When the manager assigns or routes work, it can notify an external notification service without needing to know that service's specific interface.

(2) Builder - This design pattern separates the construction of a complex object from its representation so that the same construction process can create different representations. The Builder is used to construct the workflow definition fit with its Stages (collection of Tasks) and individual Tasks. It defines the steps a workflow instance takes to go from creation to completion. *WorkflowBuilder(Builder)* is the abstract construction interface. *SequentialWorkflowBuilder(ConcreteBuilder)* constructs a definition with forward-linked stages. *EscalatingWorkflowBuilder(ConcreteBuilder)* constructs a definition with handlers in the opposite direction. *WorkflowDirector(Director)* orchestrates the construction steps. *WorkflowDefinition(Product)* is the resulting reusable workflow blueprint that defines the steps a workflow instance takes from creation to completion.

(3) Chain of Responsibility - With each role and responsibilities that staff members in the generic system comes a need for escalation. As a workflow instance moves through the system requiring approval from a staff member, in the case where a current staff member cannot approve something so it needs to be escalated to a more senior member, this design pattern avoids coupling the sender of a request by giving more than one object a chance to handle the request. The request can be passed through progressively more senior handlers. *Handler(WorkItem | Staff)* defines the common request-handling interface and has the successor link used to forward requests through the stages and tasks or between staff members. *ConcreteHandler(Stage, Task | Team, Employee)* attempts to handle the request through its responsible staff member and can pass it along the work-item chain or through checking the team's/employee's rank and forwards the request to its successor when it cannot handle it.

(4) Command - By encapsulating requests as objects, TIVIDY can support approvals and rejections as well as commands necessary for failure cases such as a retry or cancellation. It can also support undoable operations when rolling back operations is required. *Command(Command)* captures a snapshot before an action and restores it on undo, *Approve* which calls the instance's current state handler advancing pending to active or

active to completed*, Reject* which sets the workflow instance's state back to pending*, Retry* which sets pending, then calls the state handler, which advances it to active *and Cancel(ConcreteCommand)* which sets the state to cancelled*. Invoker(Invoker)* executes a command and has methods for snapshot/undo collection*, Receiver(WorkflowInstance)* holds the state and behaviour that the commands act upon*.*

(5) Composite - By composing tree-like structures that represent part-whole hierarchies we can create a generic workplace hierarchy by having all staff members as individual employees and they are grouped into teams. Additionally we can structure the steps in a workflow definition by structuring the work items to have individual tasks and stages which is a grouping of tasks. *Component(WorkItem and Staff)* lets clients interact with individual or groups of items without needing to know the internal structure*, Leaf(Task and Employee)* represents an individual item such as Register Patient or Nurse Jane Doe *and Composite(Stage and Team)* represents a group of items such as Admission, Assessment, Emergency Care or Senior Doctor*.*

(6) Facade - This design pattern provides a unified interface to a set of interfaces. By defining this higher-level interface, it makes TIVIDY and its many subsystems easier to use. As a client intending to use our system, they can initiate certain processes through the Facade without having to understand how the system functions underneath. *WorkflowFacade* gives a simple entry point to the underlying subsystem. Instead of coordinating the manager, workflow instances, definitions and strategies individually, it can call methods such as *startWorkflow(), submitWorkItem()* and *cancelWorkflow().*

(7) Memento - This design pattern was used to capture a workflow instance's state such that the object can be returned to this state later, without violating encapsulation. TIVIDY should support rollback operations should the system fail. *Originator(WorkflowInstance)* creates a snapshot and restores the saved state*, Memento(Memento)* holds the saved state reference and a work-item reference*, Caretaker(CommandHistory)* stores Memento pointers without inspecting their contents*.* Working alongside the Command, the Memento helps to support undoable operations.

(8) Observer - By defining a 1-to-many dependency between objects, the audit log and progress report observers will update themselves when the workflow instance changes its state and notifies them. TIVIDY requires monitoring, analytics and reporting and we saw it fit that this design pattern be used. *WorkflowInstance(Subject/ConcreteSubject)* maintains the observers and notifies them when relevant workflow events occur. *ProgressMonitor(ConcreteObserver)* reacts to workflow events by displaying progress information*, AuditLogObserver(ConcreteObserver)* represents an observer intended to respond to audit events and check the event log. *Observer(Observer)* defines the interface that objects implement to receive workflow event notifications.

(9) State - A workflow instance goes through a lifecycle starting at creation and ending at completion. An object's behaviour should be modelled such that the object's behaviour can

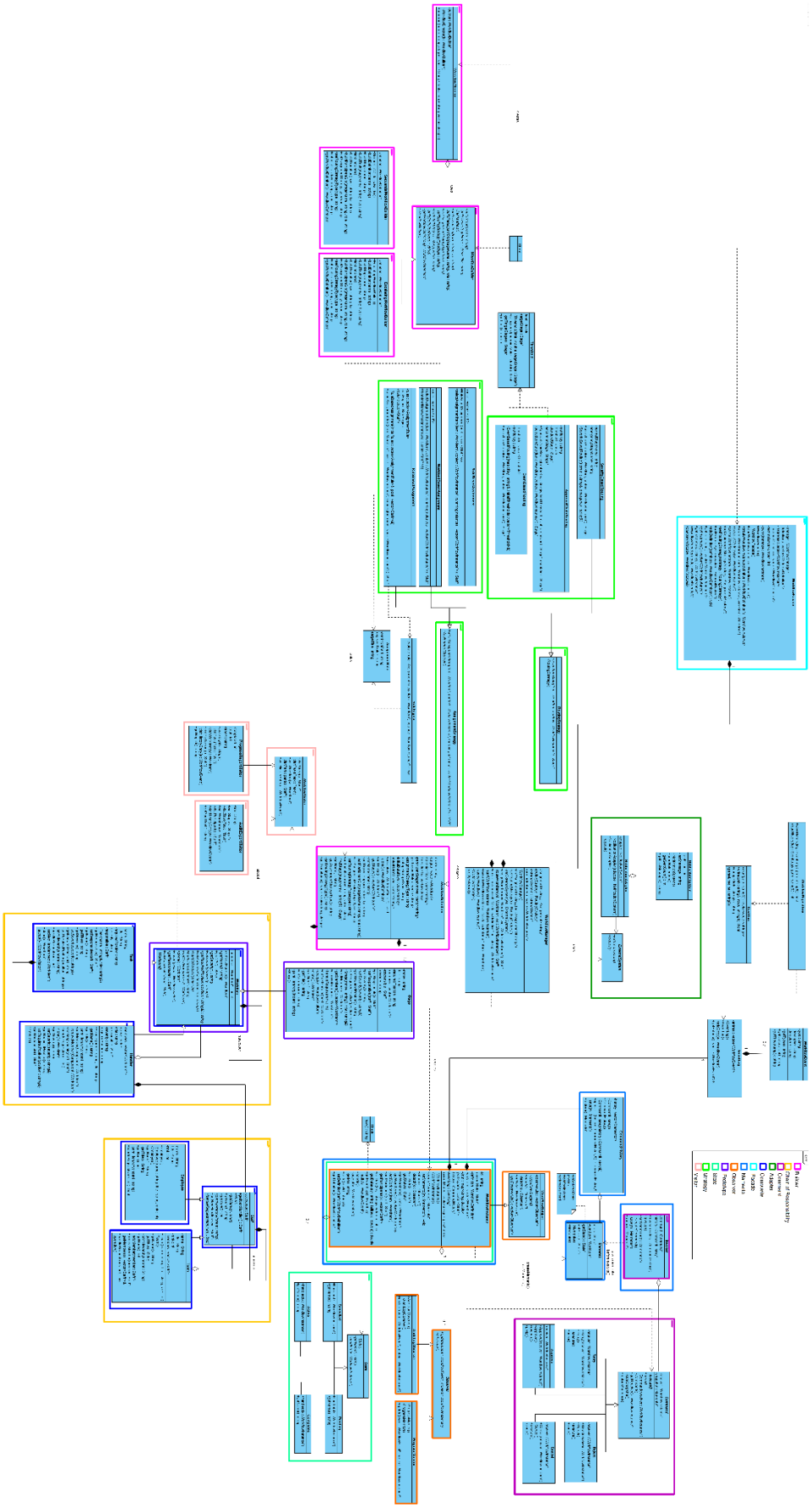
change at runtime and it appears to be changing its class. *Pending, Active, Completed and Cancelled (ConcreteState)* implement the runtime behaviour of a workflow instance - certain restrictions may be placed on an object dependent on its state. *State(State)* declares an interface for encapsulating the workflow instance's behaviour. *WorkflowInstance(Context)* maintains an instance of either of the 4 concrete states.

(10) Strategy - A workflow management system requires routing and assignment rules. This design pattern helps to fill that by having a family of algorithms that are interchangeable dependent on the context. The *AssignmentStrategy(Strategy)* determines who should handle a work item. The *RoutingStrategy(Strategy)* determines which stage should receive the work item. *RoleBasedAssignment, RuleBasedAssignment and WorkloadBasedAssignment(ConcreteStrategy)* implement the *AssignmentStrategy's* interface with staff members being selected for handling a work item by their role or the staff with the least amount of active cases or a rule based strategy. *SeverityBasedRouting, ApprovalChainRouting and EventBasedRouting* implement the *RoutingStrategy's* interface and decide which stage should next receive a work item. This is either through the severity of an instance - in our hospital it could be an emergency patient needing to go straight to emergency care thus skipping admission and assessment possibly. It could be an approval chain based routing where an instance is sent through to a stage based on the approval it gets from a staff member. It could be based on events taking place in the system and by using an eventMetric an instance can be routed to a particular stage.

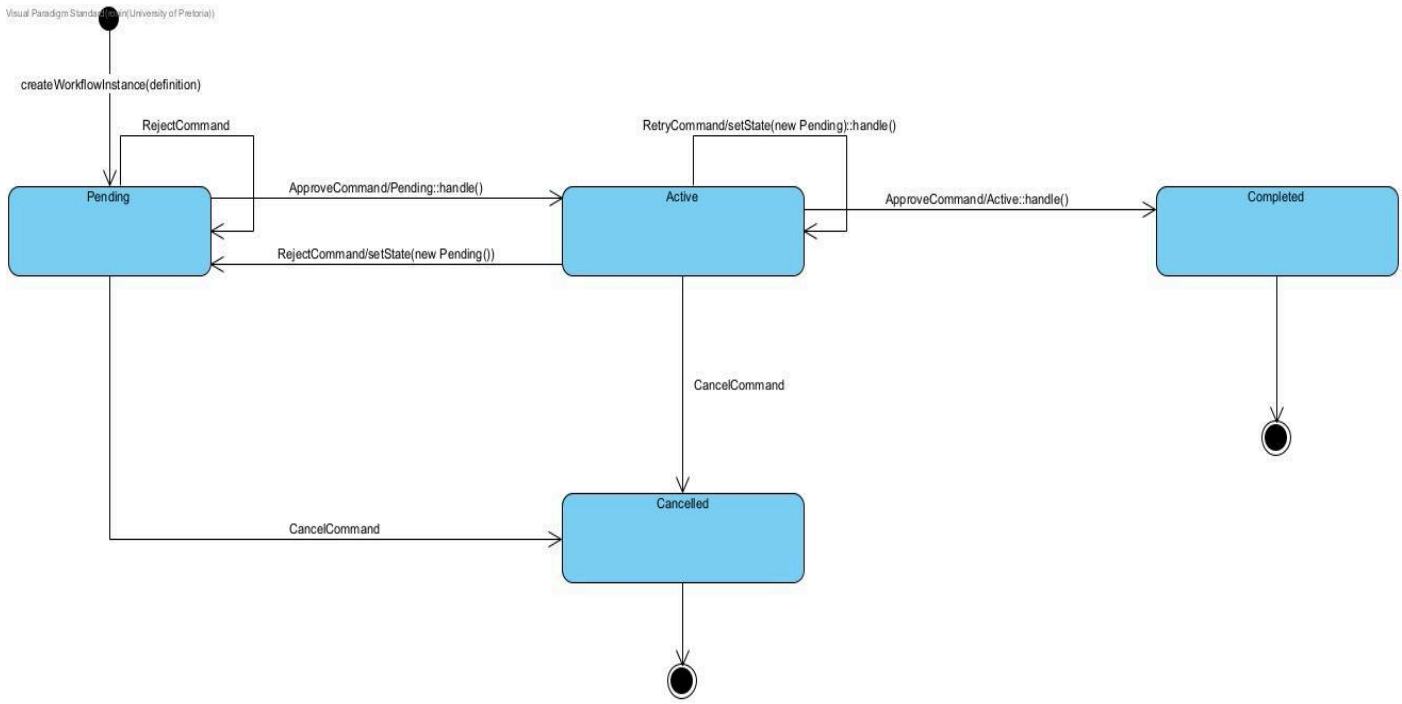
(11) Visitor - This design pattern separates algorithms from the structures they operate on. Allowing us to add new operations to various element types without modifying their source code. *Element (WorkItem)* defines the *accept()* interface through which a visitor can visit a work item*, ConcreteElement(Stage, Task, Staff, WorkflowEvent)* accepts a visitor for all the various element types. *Visitor(WorkflowVisitor)* defines visit operations for the supported element types. *ConcreteVisitor(ProgressReportVisitor, AuditExportVisitor)* implements progress-report and audit-export related operations for visited elements.

(12) Prototype - clone of workflowinstance. Instead of having to create an object from scratch, you can clone a workflowInstance object instead. This makes sure that we can copy existing objects without making code dependent on their classes. This also means that changes in one instance will not unintentionally affect other instances, as we are cloning objects.

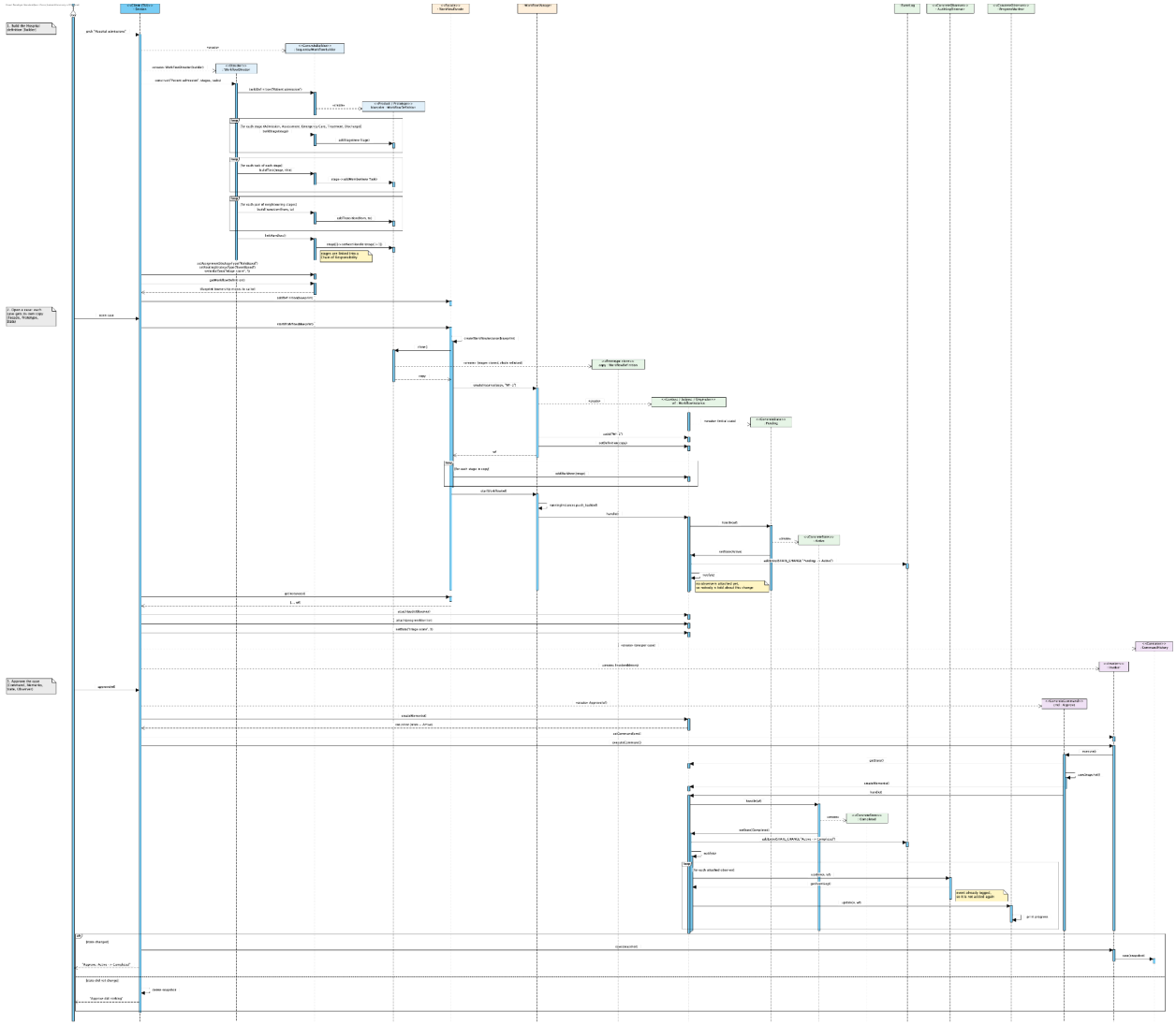
Task 5: UML Class Diagram



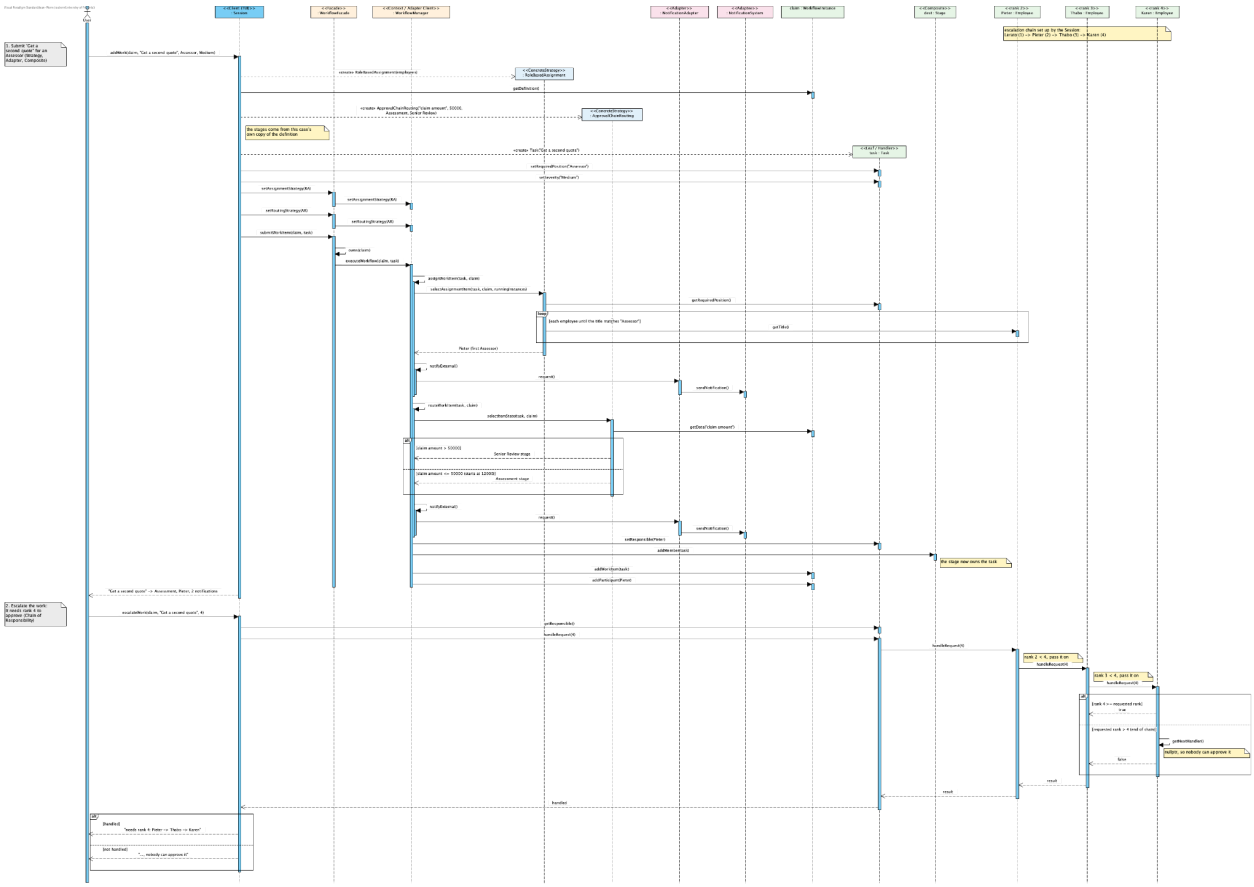
Task 6: Behaviour Runtime Diagrams



State Diagram- State of a WorkflowInstance



Sequence Diagram 1- Hospital: build the definition, open a case, approve it



Sequence Diagram 2- Insurance: submit work to a claim, then escalate it for approval

Task 7: Design Decisions and Revisions

Our initial design focused on integrating our research on workflow systems with our knowledge of the Gang of Four design patterns for software modeling. But as we implemented our design, we made numerous changes to our initial concept to ensure that we produced a well-structured and efficient system that met all requirements.

The first notable difference is the **role of the WorkflowManager class**. In our previous implementation, this class was described as the central coordinator of the workflow execution, with it being used by at least 8 different design patterns - such as invoking commands, managing mementos, selecting strategies and initiating escalations. We felt that while this was efficient in terms of centralizing the system, it resulted in too tight of a coupling between the classes. In our revised implementation, this class has a more specified role, in being the client for the Adapter pattern and coordinator through the Facade. This change helped define a stricter separation of concerns.

Another important difference is the **addition of the Builder design pattern**. In our initial implementation, there was no Builder pattern present, and Workflow Definitions were not defined as separate to Workflow Instances. But with the Builder pattern, we were now able to separate the construction of these definitions from its representation, and allow for different types of workflows to be constructed, which integrated well with other parts of our system.

In addition, our **implementation of the Chain of Responsibility** design pattern changed quite a bit. Previously, our classes were very vague, with a ‘Handler’ class, and ‘SupportHandler’ classes as our concrete handlers. And while our design was meant to be generic, it was not clear what responsibilities were being passed on, and to whom. In our revised design, we were able to integrate Chain of Responsibility with our Composite pattern implementations, such that there was a clear hierarchy of classes and to whom a responsibility would be passed on to in each instance. The passing on of responsibilities is now determined by if a member has enough authority to approve a request.

Our **Composite implementation** also differs from the initial to revised design. Before, we had one Composite structure, pertaining to a Supervisor and Employee hierarchy, with basic tree-building methods. But in our final design, we implemented two separate hierarchies; for WorkItems and for Staff, which work in conjunction with Chain of Responsibility to implement a WorkItem being passed down through the two hierarchies.

The **implementation of Memento** differs slightly, but it has a huge impact on our system. In our previous implementation, the WorkflowManager acted as the Caretaker for the system, storing instances of the memento for WorkItems and States. However, in our revised implementation, CommandHistory acts as the Caretaker, relieving WorkflowManager of some of the work within the system, and ensuring separation of concerns whilst integrating the design patterns.

Another important difference is the **class responsible for being the Subject in the Observer pattern**. In our initial implementation, the WorkflowManager was once again responsible for coordinating notifications for workflow events between the classes, but in our new design, the WorkflowInstance class is now in charge of notifying the observers of state changes, and acting as the Subject participant. Again, this relieves WorkflowManager of coordinating most of the work for the system.

A **pattern which was present in our previous design but not current is Iterator**. Previously, it was used to explicitly traverse collections of WorkItems in ways deemed useful to the system, but it was subsequently removed and replaced by C++'s built-in iteration used in std::vector. This helped to eliminate unnecessary and pointless classes in our design and resulted in a more efficient system.

Along with the removal of Iterator, the **Visitor pattern was added** to our system. This pattern allows for operations to be performed on workflow objects without modifying the classes themselves. This helps separate methods from the classes to ensure that every other design pattern implements its primary responsibilities in its classes, and additional operations are stored in the Visitor.

Our final revision was a change to the **C++ standard used by the project**. The specification asks for C++11, but to give TIVIDY a full-screen, interactive TUI we used FTXUI, a terminal UI library that requires C++17. After our tutor confirmed that this was acceptable, we moved the whole project from C++11 to C++17. CMake now downloads FTXUI and GoogleTest itself, so no extra installation is needed. Only the TUI program links FTXUI; the workflow engine and the unit tests do not depend on it, so the engine stays independent of the user interface.

Task 8: Industry Scenarios

The first scenario, involving the Hospital Patient Admissions and Treatment Workflow.

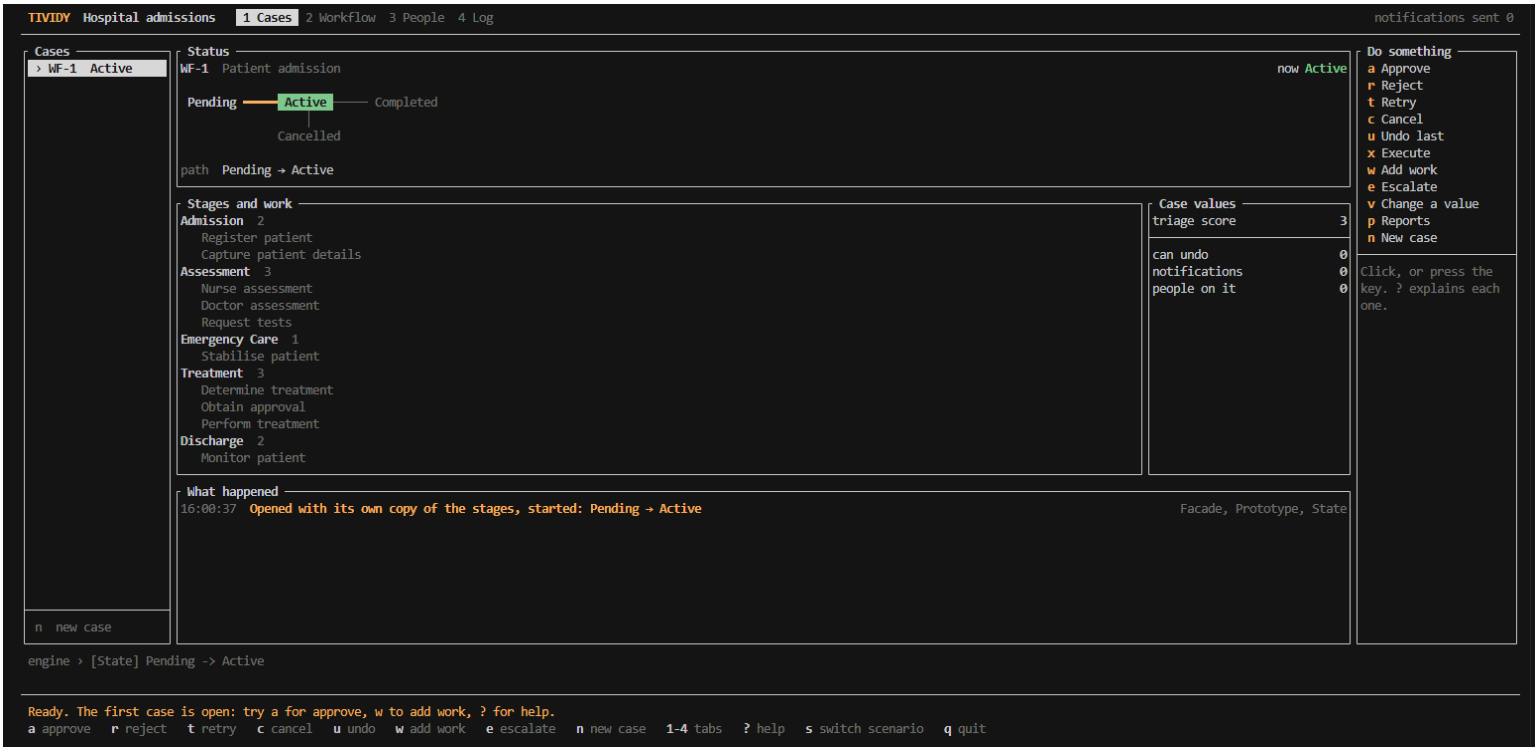
Initially, the TUI opens with **3 teams** and **5 employees** established - namely Ann Botha in the Admissions team (Front Desk), Jane Mokoena and Sipho Dlamini in the Nursing team (Ward), and Dr Priya Naidoo and Dr Mark Steyn in the Medical team (Doctors).

There are also **5 stages** and **11 tasks** which are predetermined - namely, **Admission stage**, with "Register patient" and “Capture patient details”, **Assessment stage**, with “Nurse assessment", “Doctor assessment”, and “Request tests”, **EmergencyCare stage**, with “Stabilise patient”, **Treatment stage**, with “Determine treatment", “Obtain approval”, and “Perform treatemnt”, and **Discharge stage**, with "Monitor patient” and “Discharge patient".

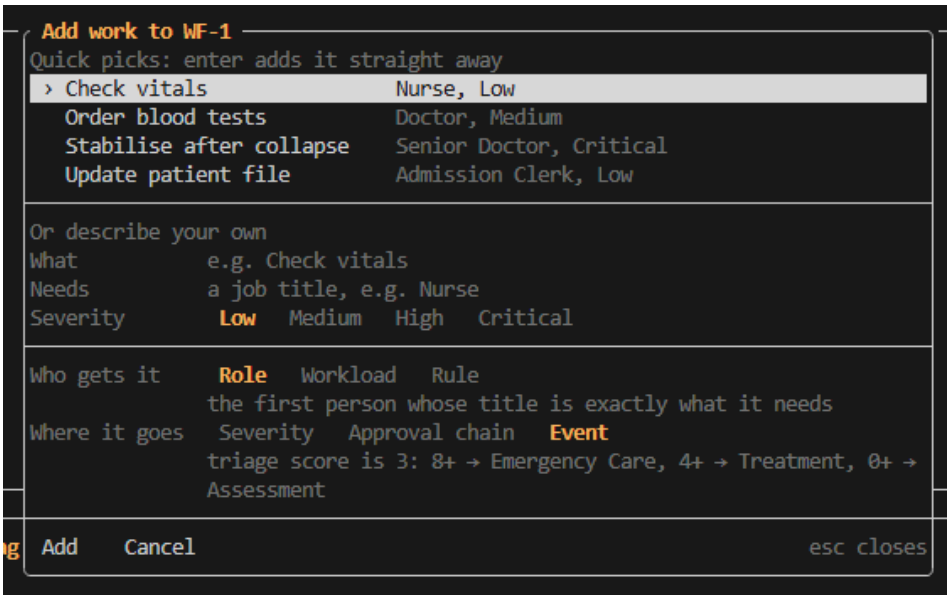
The scenario is configured to make use of a **RoleBased Assignment Strategy** and **EventBased Routing Strategy**. It also uses **SequentialWorkflowBuilder** to construct (build) the workflow.

It also assigns **ranks** to the employees, which are used by **Chain of Responsibility** to determine whether an employee has sufficient authority to handle a patient escalation request, or if the request must be passed on. The escalation chain follows Ann Botha → Jane Mokoena → Dr Priya Naidoo → Dr Mark Steyn

When the scenario is established, **one patient workflow instance** is automatically created in the **Active state**, with an initial triage score of 3 - which corresponds to the **Assessment stage** according to the predetermined **EventBased Routing Strategy**.



Instruction 1 - Add work to the patient case



‘w’ is pressed to open the **‘Add Work’** form.

Quick pick **‘Check vitals’** was selected by pressing **‘Enter’**

This new task now appears in **‘Stages and work’**

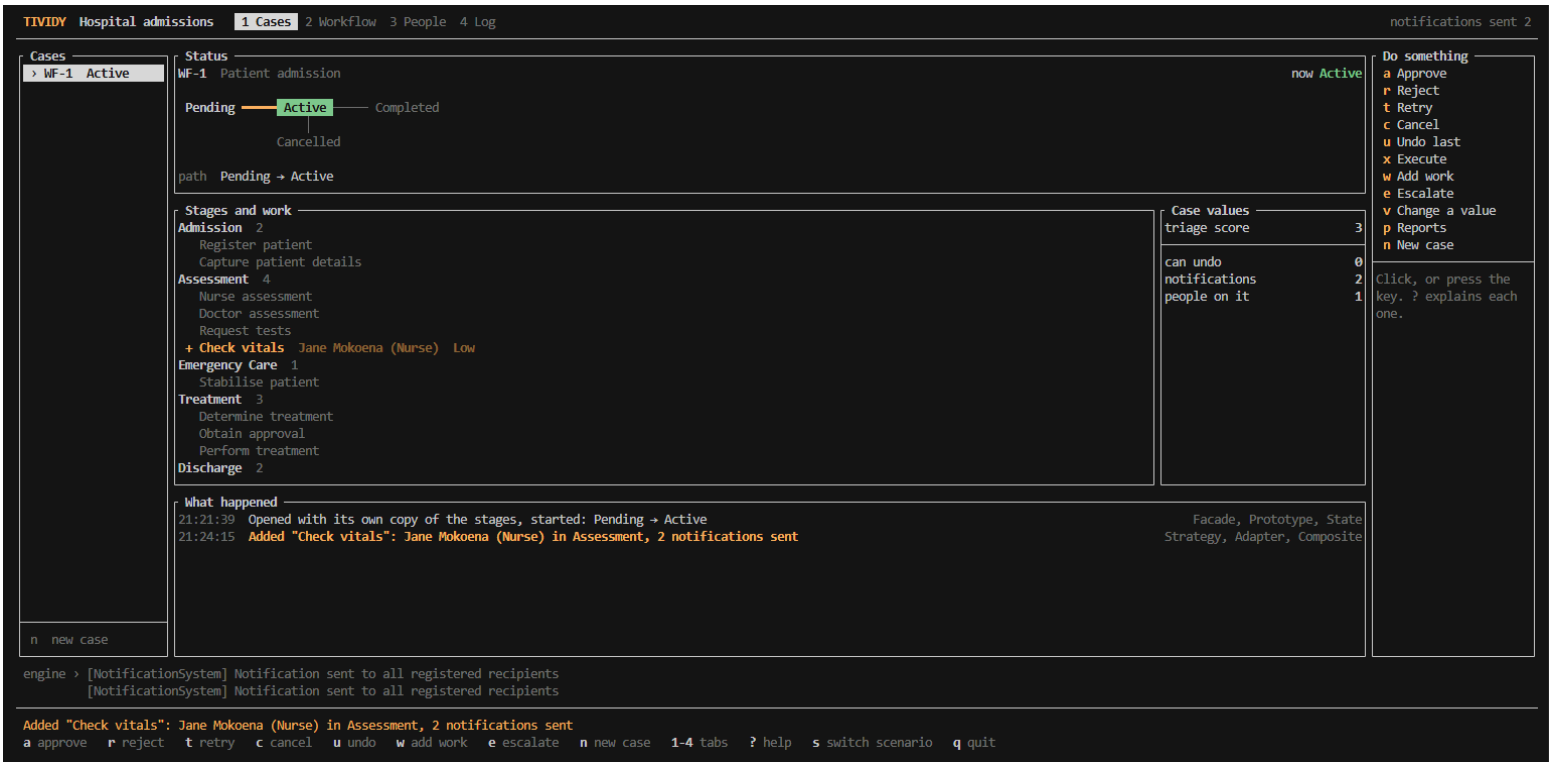
Because the triage score is 3, the **EventBased Routing Strategy** directs the task to the **Assessment stage.**

The **RoleBased Assignment Strategy** selects an eligible **Nurse.**

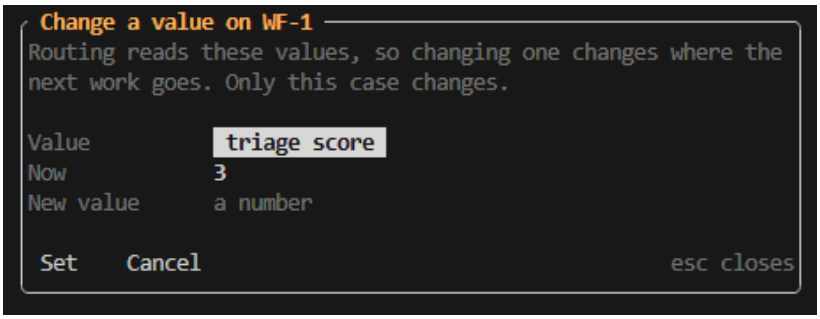
The **‘What happened’** timeline records the operation.

The number of people on the case and notifications increased.

This instruction made use of the **Strategy** and **Observer** patterns.



Instruction 2 - Change the triage score



‘v’ is pressed to open the **‘Change a value’** form.

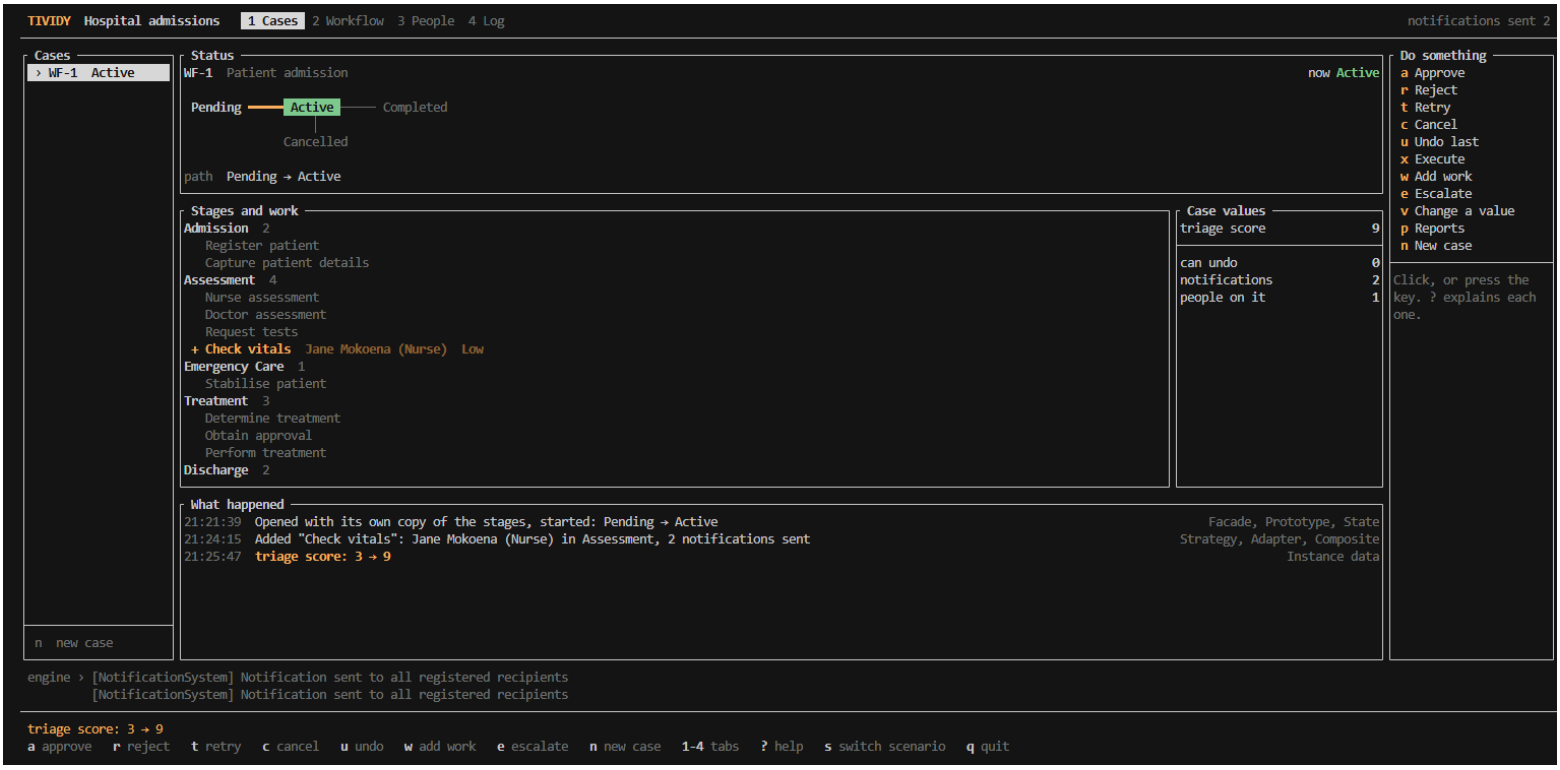
9 was entered as **‘New value’** and **‘Enter’** was pressed.

The **‘Case values’** panel changes from **triage score 3** to **triage score 9**.

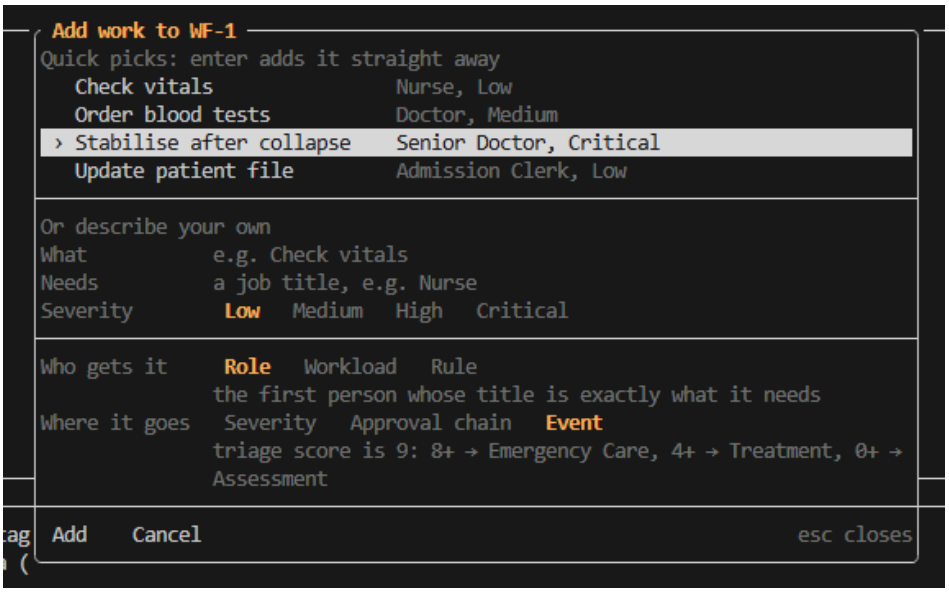
The **‘What happened’** timeline records the change.

The case remains **‘Active’**, but a triage score of 9 is above the set threshold of 8, so subsequently submitted work should be routed to **‘Emergency Care’**.

This instruction made use of the **Strategy** pattern.



Instruction 3 - Add emergency work



‘w’ is pressed to open the **‘Add Work’** form.

Quick pick **‘Stabilise after collapse’** was selected by pressing **‘Enter’**

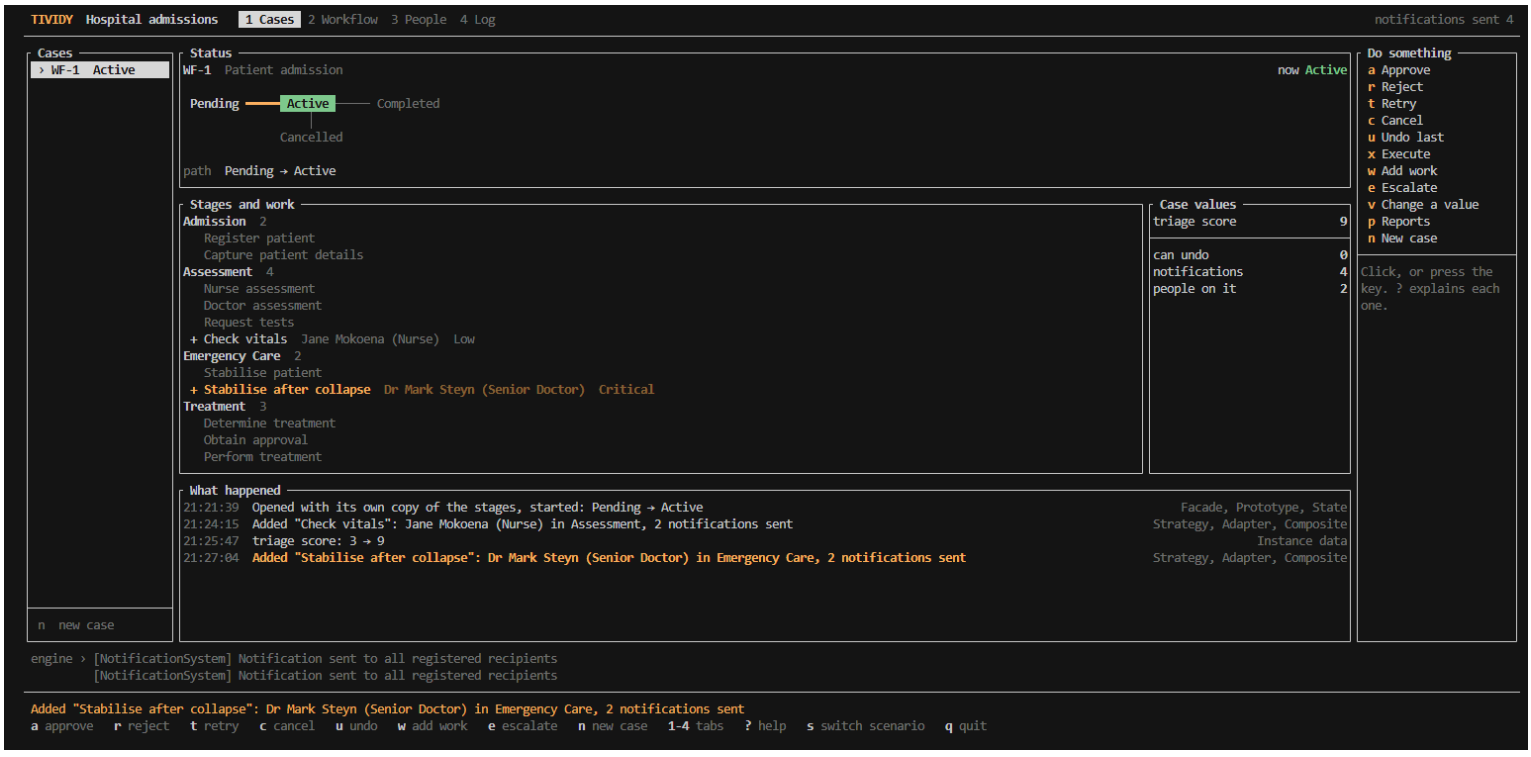
‘Stabilise after collapse’ appears under **‘Emergency Care’**.

The **RoleBased Assignment Strategy** selects **‘Dr Mark Steyn’**, the predefined senior doctor.

The task shows it is assigned **‘employee’** and **‘Critical severity’**.

The **‘What happened’** timeline records the operation.

The instruction made use of the **Strategy** and **Observer** patterns.



The second scenario, involving the Insurance Claim

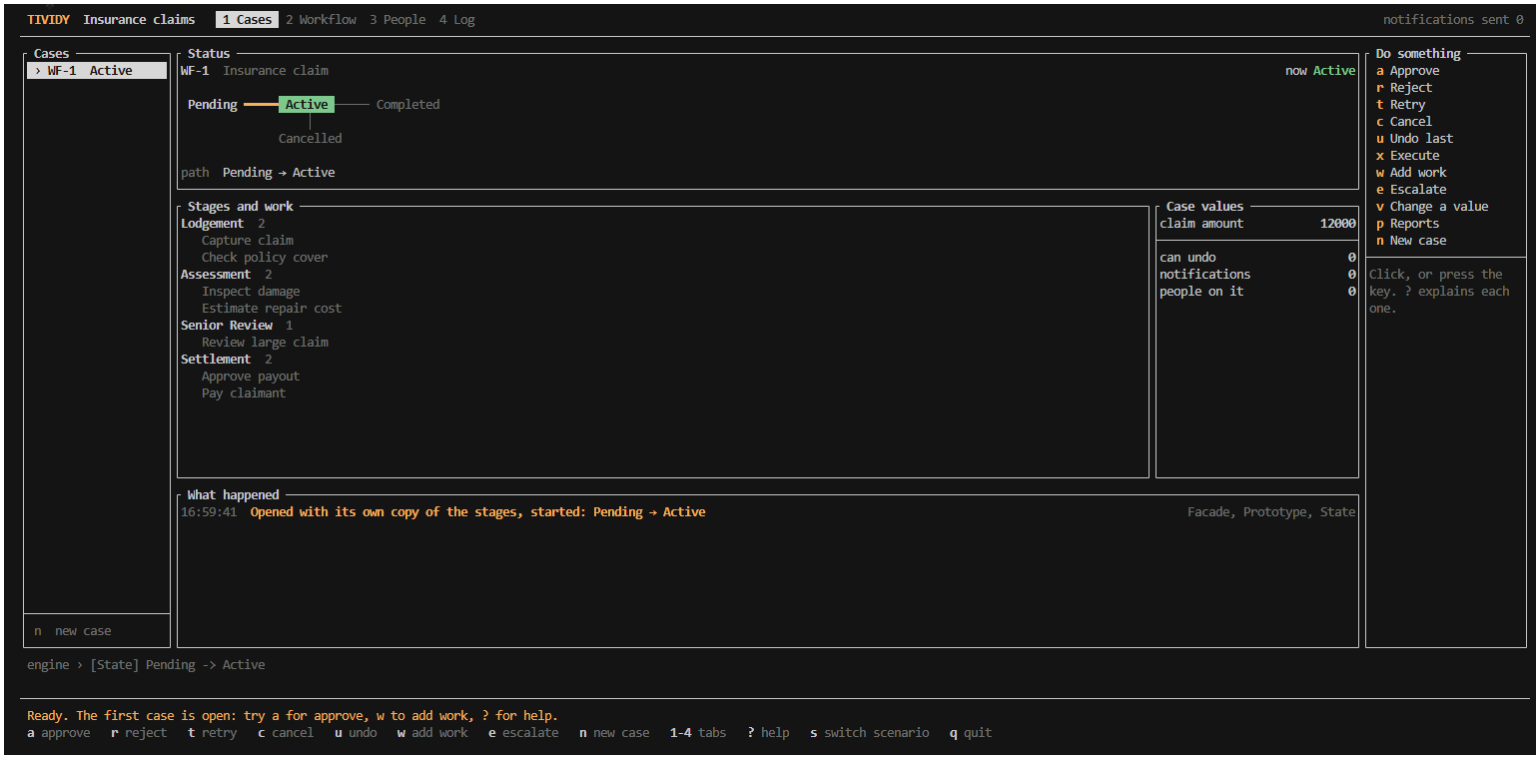
Initially, the TUI opens with **3 teams** and **5 employees** established – namely Lerato Khumalo in the Claims team (Claims Desk), Pieter van Wyk, Aisha Patel and Thabo Nkosi in the Assessment team (Assessors), and Karen Smit in the Management team (Management).

There are also **4 stages** and **7 tasks** which are predetermined – namely, the **Lodgement stage**, with “Capture claim” and “Check policy cover”, the **Assessment stage**, with “Inspect damage” and “Estimate repair cost”, the **Senior Review stage**, with “Review large claim”, and the **Settlement stage**, with “Approve payout” and “Pay claimant”.

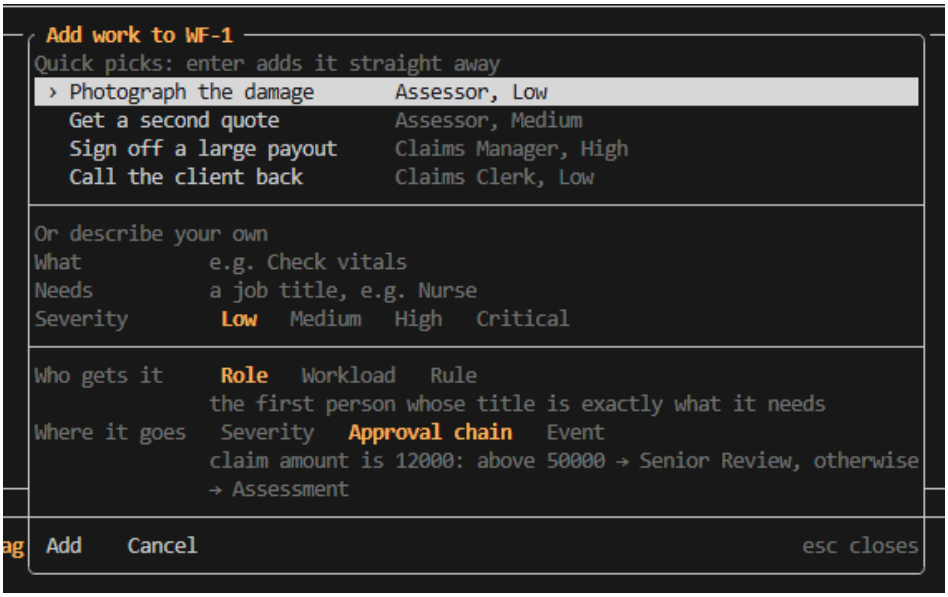
The scenario is configured to make use of a **RoleBased Assignment Strategy** and **ApprovalChain Routing Strategy**. It also uses **EscalatingWorkflowBuilder** to construct (build) the workflow.

It also assigns **ranks** to the employees, which are used by the **Chain of Responsibility** pattern to determine whether an employee has sufficient authority to handle an insurance claim escalation request, or whether the request must be passed on. The escalation chain follows Lerato Khumalo → Pieter van Wyk → Thabo Nkosi → Karen Smit.

When the scenario is established, **one insurance claim workflow instance** is automatically created in the **Active state**, with an initial **claim amount of R12,000**. According to the predetermined **ApprovalChain Routing Strategy**, claims exceeding the **approval limit of R50,000** are directed to the **Senior Review stage**, while claims within the limit are directed to the **Assessment stage**. Therefore, the initial claim amount of R12,000 corresponds to the **Assessment stage** when work is routed.



Instruction 1 - Add work to the insurance claim



‘w’ is pressed to open the **‘Add Work’** form.

Quick pick **‘Photograph the damage’** was selected by pressing **‘Enter’**

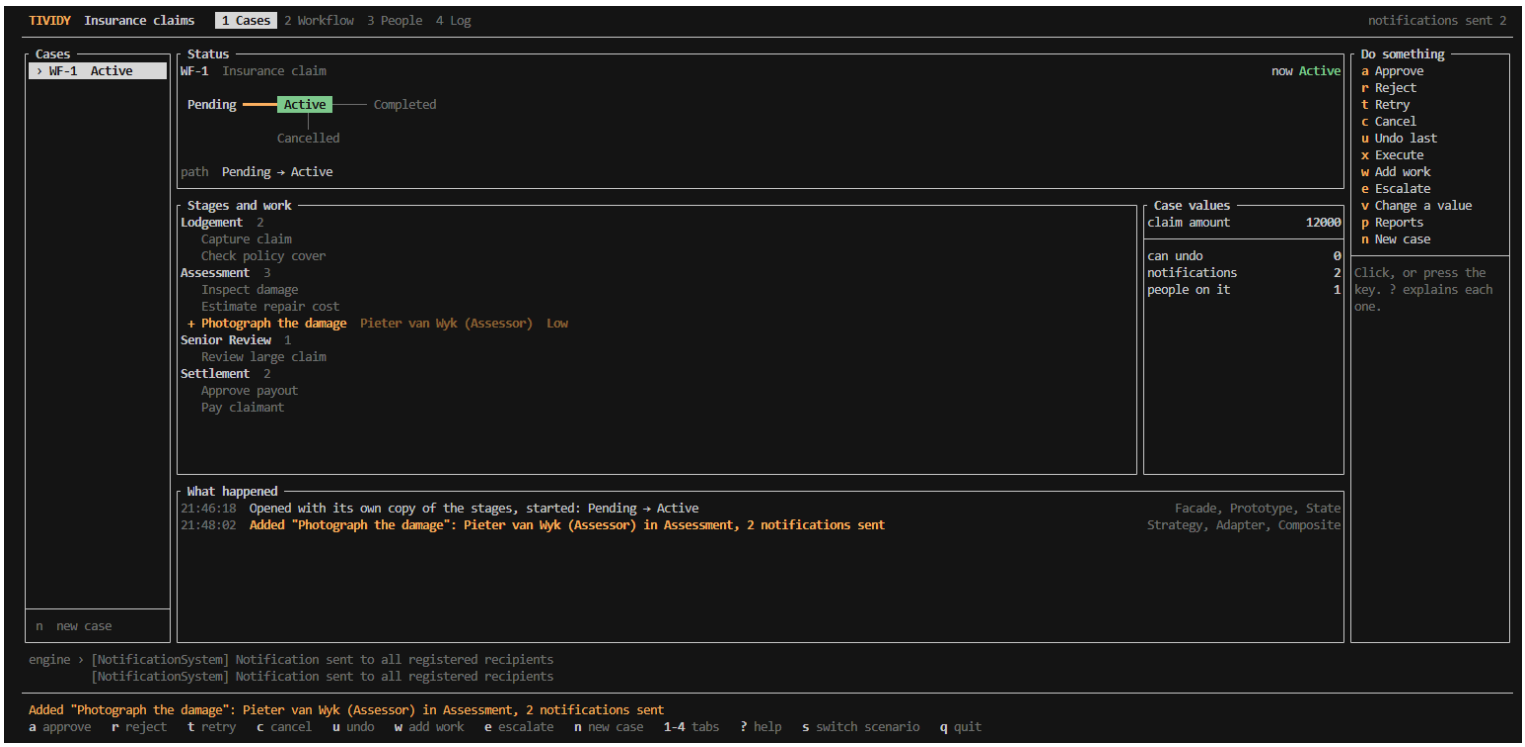
Because the triage score is 3, the **EventBased Routing Strategy** directs the task to the **Assessment stage.**

The **RoleBased Assignment Strategy** selects an eligible **Assessor**, such as Pieter van Wyk.

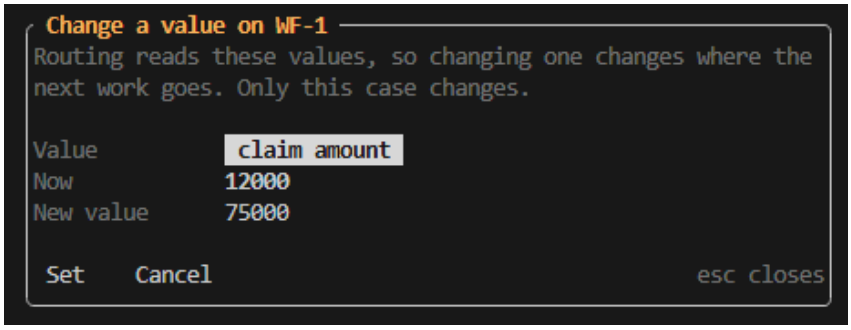
The new task appears with its assigned employee and severity.

The **‘What happened’** timeline records the operation.

This instruction made use of the **Strategy** and **Observer** patterns.



Instruction 2 - Increase the claim amount



‘v’ is pressed to open the **‘Change a value’** form.

75000 was entered as **‘New value’** and **‘Enter’** was pressed.

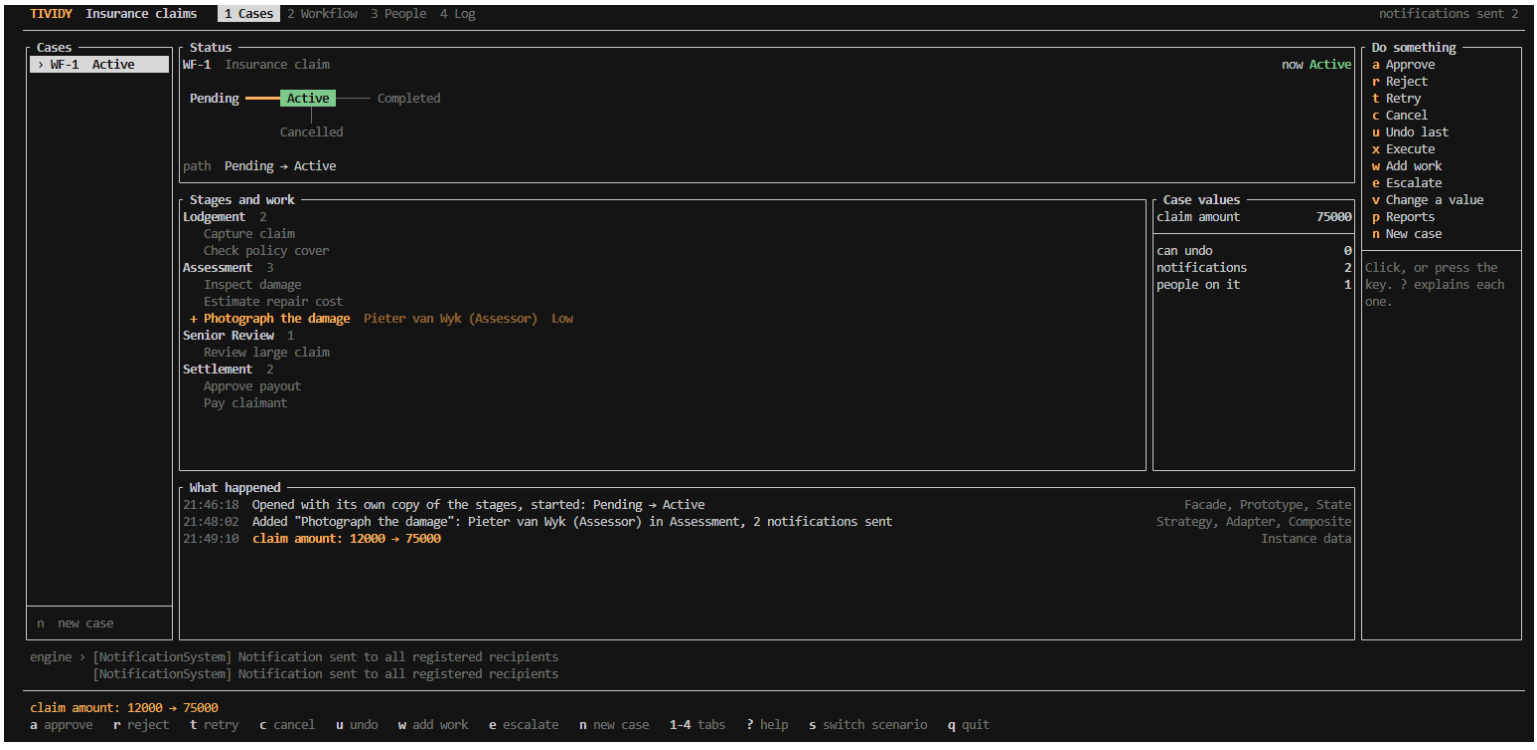
The **‘Case values’** panel changes from **claim amount 12000** to **claim amount 75000**.

The claim remains **‘Active’**.

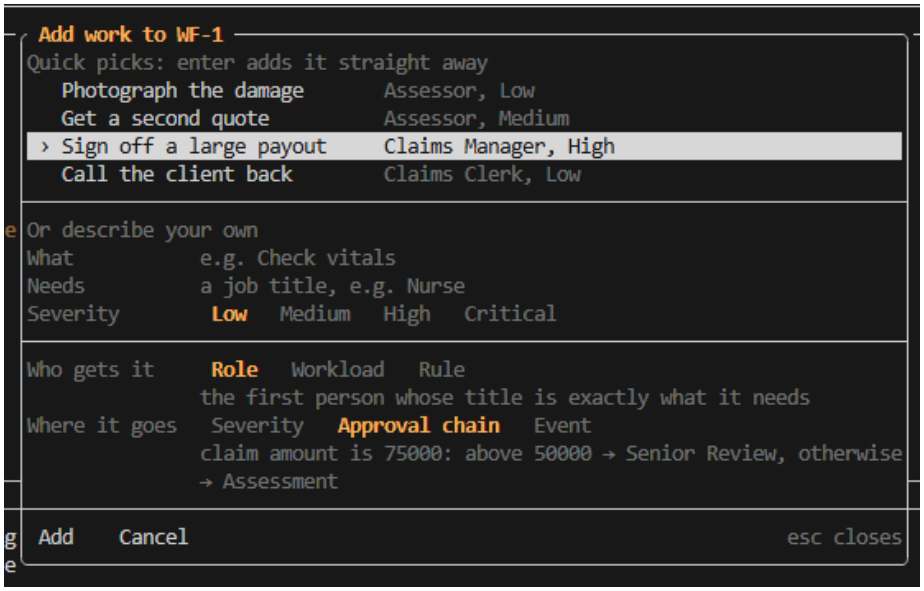
The **‘What happened’** timeline records the change.

Future work should now be directed to **Senior Review**, because 75000 exceeds the 50000 approval limit.

This instruction made use of the **Strategy** pattern.



Instruction 3 - Add work requiring senior approval



‘w’ is pressed to open the **‘Add Work’** form.

Quick pick **‘Sign off a large payout’** was selected by pressing **‘Enter’**

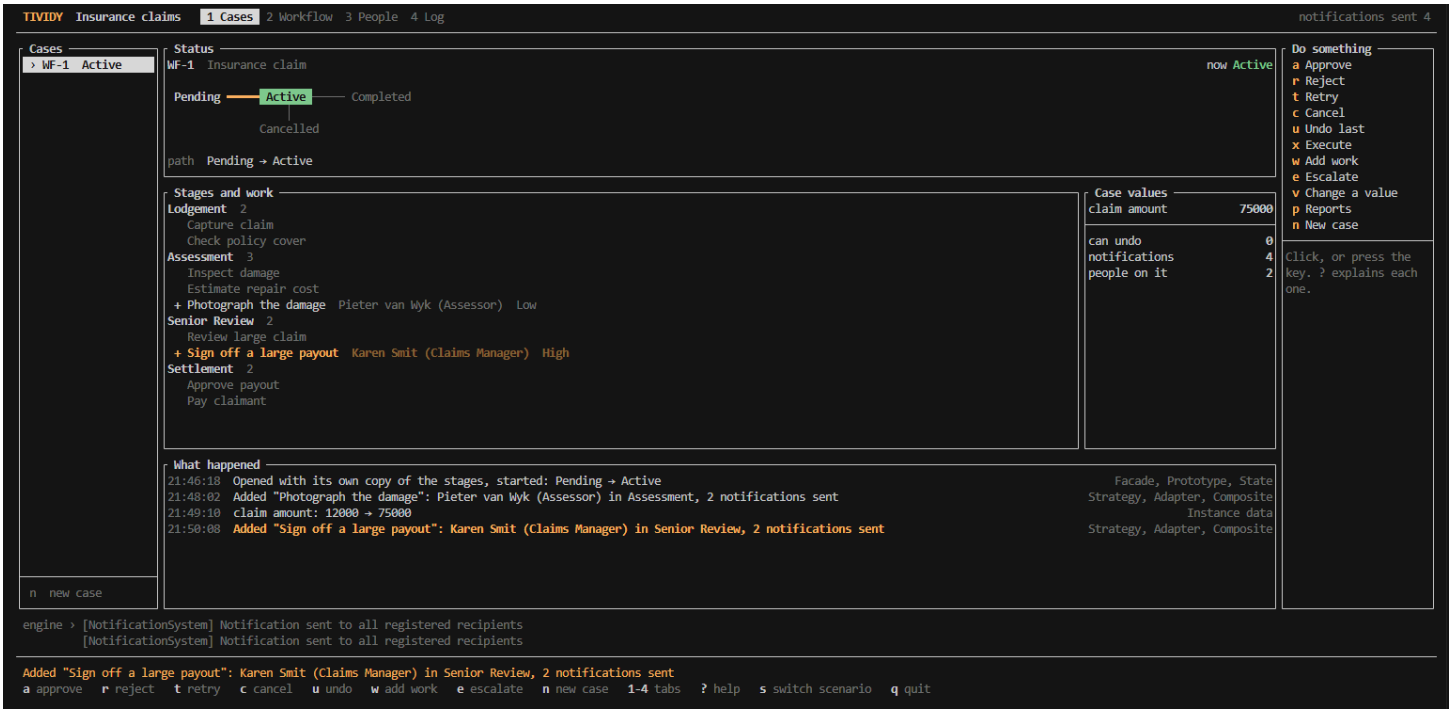
‘Sign off a large payout’ appears under **‘Senior Review’**.

The **RoleBased Assignment Strategy** selects **‘Karen Smit’**, the predefined Claims manager.

The task shows it is **‘assigned employee’** and **‘High severity’**.

The **‘What happened’** timeline records the operation.

The instruction made use of the **Strategy** and **Observer** patterns.



Build / Run / Test Instructions:

Requirements for local execution

The project requires:

- g++ (C++17)
- cmake (3.14 or newer)
- make

CMake downloads FTXUI (the TUI library) and GoogleTest on the first build, so they do not need to be installed. Run the TUI in a terminal of at least 120 x 36; press ? for help and q to quit.

Building and running locally

```
make          # Build the project
make run      # Build and open the TUI
make test     # Build and run the test suite
make clean    # Remove build output
```

Using Docker

Alternatively, the project can be built and run using Docker:

```
docker compose build              # Build the image
docker compose run --rm tividy    # Open the TUI
docker compose run --rm tests     # Run the test suite
```

The first build takes a few minutes while CMake, FTXUI and GoogleTest are set up in the image. docker compose run does not rebuild the image, so run docker compose build again after changing the code.

Generating documentation

Doxygen documentation can be generated using:

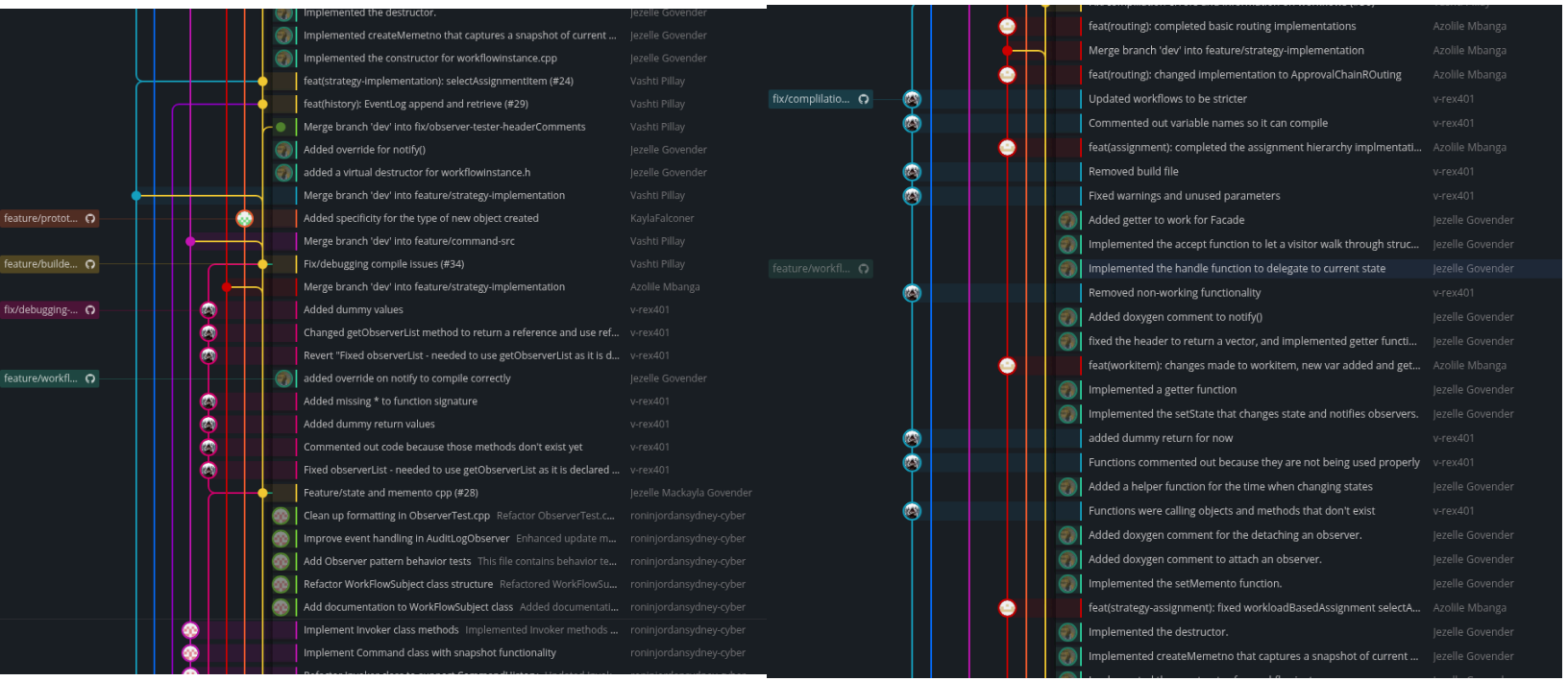
```
doxygen Doxyfile
```

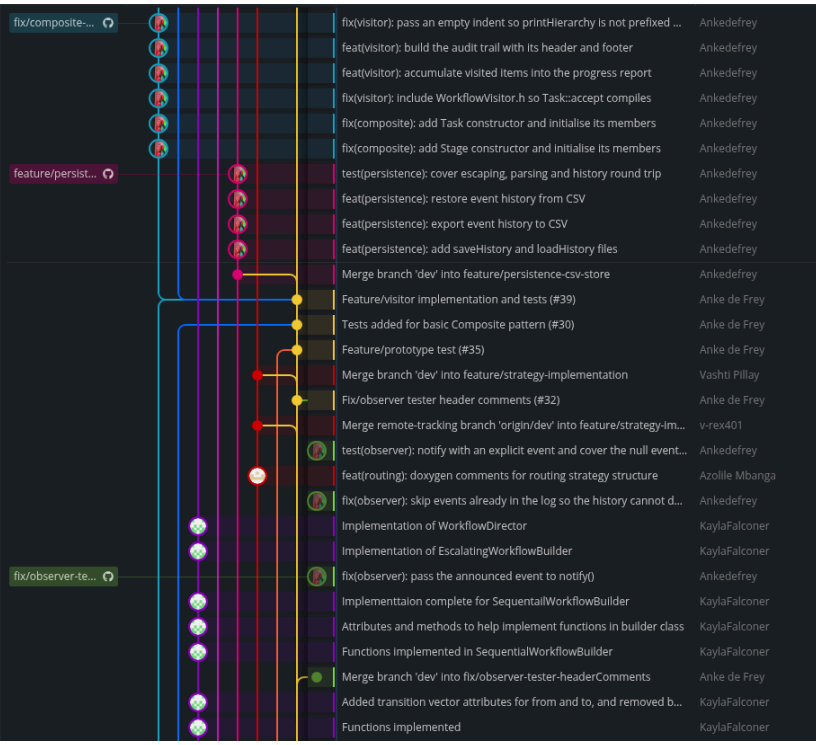
The generated documentation is available at docs/html/index.html

Relevant Git and Engineering Evidence

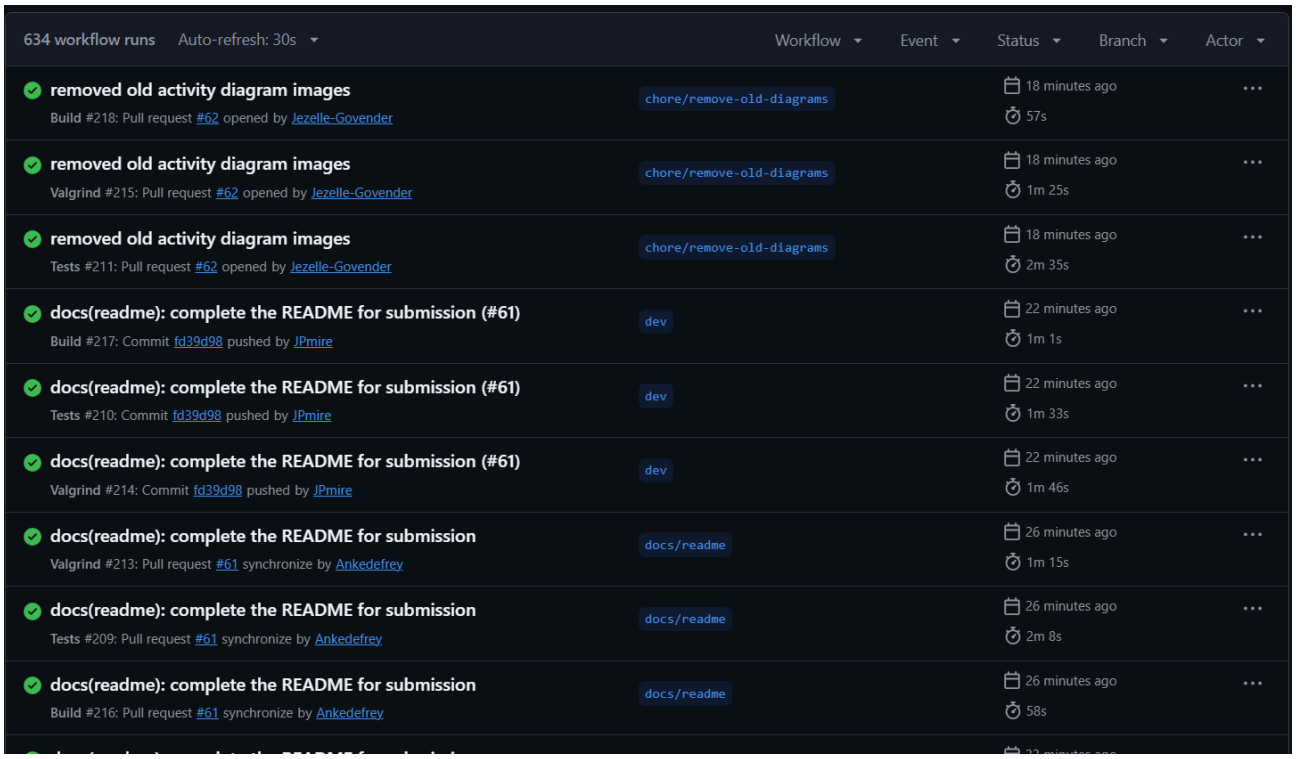
Possibly a photo of GitHub Insights?

Evidence of Branching and Github development

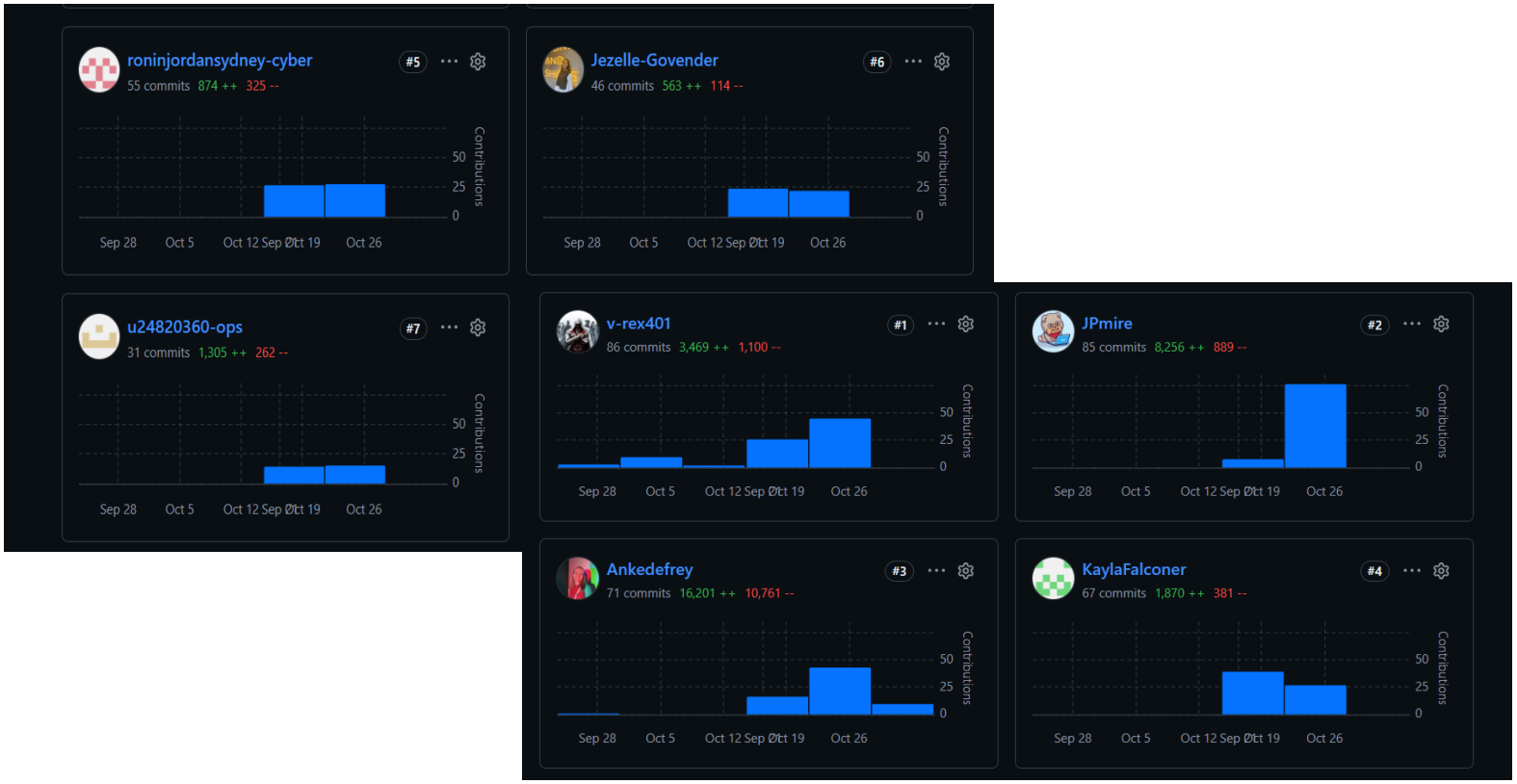




Workflow Runs



Contributions



Team Contribution Statement

Azolile Mbanga helped design the first draft of the UML class diagram. He programmed the WorkflowManager and Strategy pattern subsystem. He helped to complete the final PDF document.

Anke de Frey implemented the Adapter pattern and the external notification integration, the event log and workflow history subsystem, and the CSV persistence layer. Kept the dev branch building and tested together with Vashti. Contributed to final design of the UML class diagram.

Vashti Pillay implemented the Visitor Pattern, namely the WorkflowVisitor, ProgressReportVisitor and AuditReportVisitor. Set up the rulesets and workflows of the github to protect our main branch and dev branch from accidental pushes and set up rules to protect against code that did not compile or had memory leaks. Contributed to the final design of the UML class diagram, main.cpp code and scenarios. Managed github and tested with Anke.

Kayla Falconer helped design the first draft of the UML class diagram. Programmed the Builder, Chain of Responsibility and one of the Composite design pattern implementations. Helped to complete the final PDF document.

Jean-Pierre Joubert designed the initial sequence diagrams. Implemented the WorkflowFacade and developed the FTXUI TUI.

Ronin Sydney designed the Activity Diagram 3 and the State Diagram. Implemented the Code for all of the Command and Observer pattern classes/functions in the design

Jezelle Govender designed the Activity Diagram 1 and Activity Diagram 2. Implemented the code for State and Composite pattern, and WorkflowInstance (Central Coordinator) in the design.